

Codeforces 449B

Jzzhu and Cities

Dijkstra com análise de arestas redundantes

Grupo F

Integrantes: João Vitor Silva • Antonio Davi • Pablo Dornelles

Orientador: Prof. Me Ricardo Carubbi

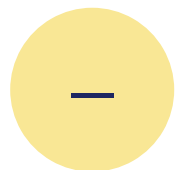
Linguagem: Python 3 • Plataforma: Codeforces

1. O problema



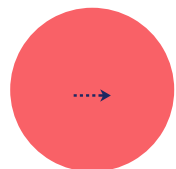
n

idades
(capital = 1)



m

estradas
bidirecionais



k

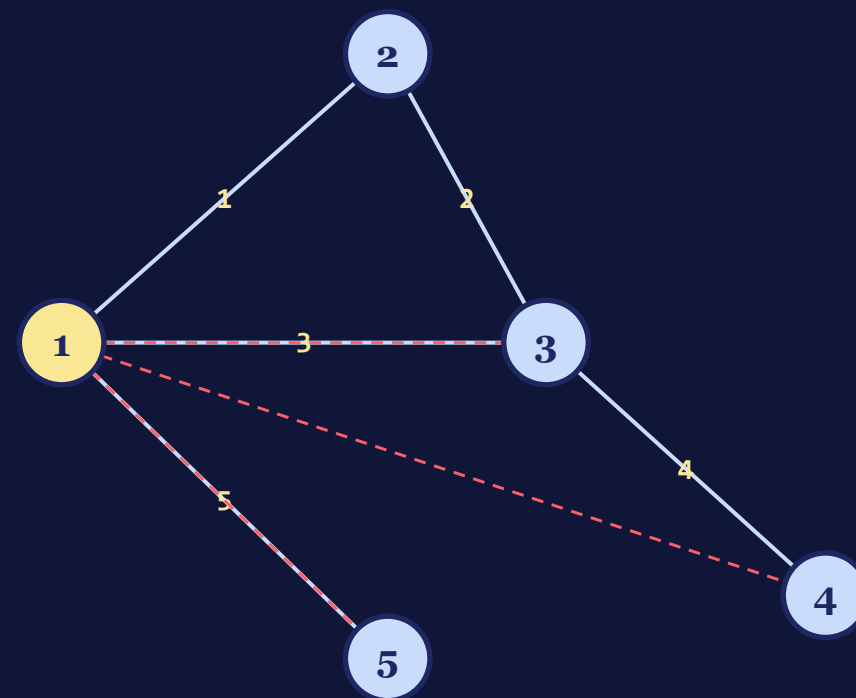
trens
da capital

Objetivo: fechar o máximo de trens sem aumentar a distância da capital a nenhuma cidade.



trens podem ser fechados (saída do exemplo 1)

Exemplo 1



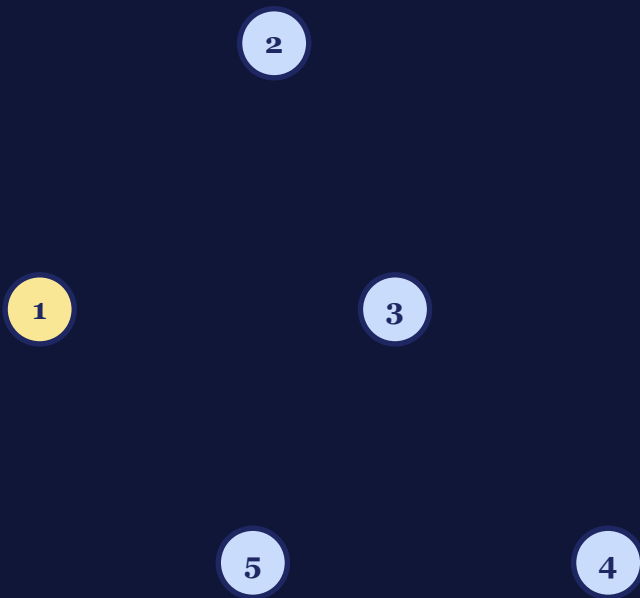
— estrada

- - - trem (da capital)

2. Do enunciado ao grafo

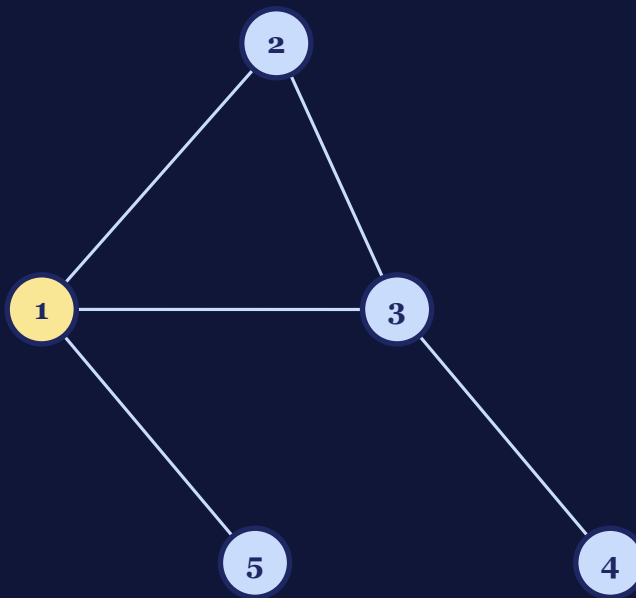
A imagem cresce em três etapas:

1. Cidades → vértices



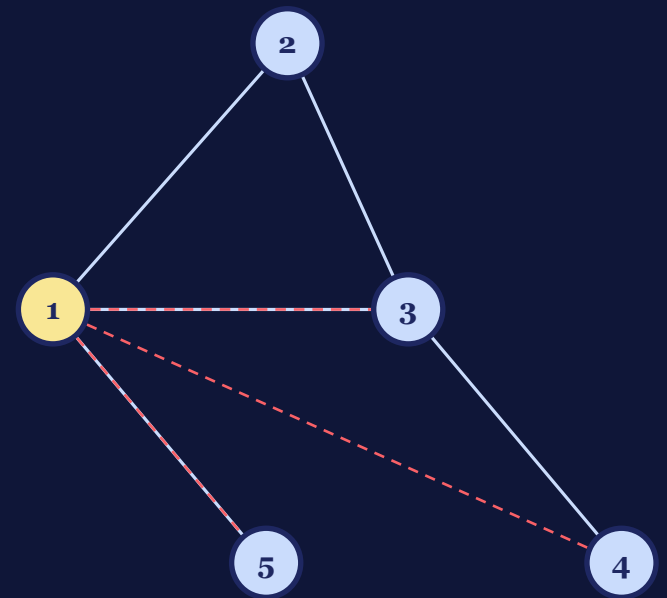
n nós; cidade 1 é a capital.

2. Estradas → arestas



Aresta bidirecional de peso x.

3. Trens → arestas extras



Aresta 1 → s marcada como trem.

3. Modelagem como grafo

Estrada (u, v, x)



aresta bidirecional, peso x

Trem (s, y)



aresta extra $1 \rightarrow s$, peso y

Menores distâncias



Dijkstra de origem única
(vértice 1)

Por que Dijkstra?

≥ 0

Pesos não negativos
($x, y \geq 1$)

1

Origem única:
a capital

+

Trens são só
arestas extras

$\text{adj}[u] = [(\text{vizinho}, \text{peso}, \text{é_trem})]$

$V = n \quad \bullet \quad E = 2m + k$

4. Estratégia: Dijkstra + via_estrada

Varição: uma única execução de Dijkstra, com uma anotação extra por vértice — sem expansão de estado.

Relaxar $u \rightarrow v$



$$nd = dist[u] + w$$

$nd < dist[v] \rightarrow dist[v]=nd, via_estrada[v] = (\text{não é trem})$

$nd = dist[v] \text{ e } estrada \rightarrow via_estrada[v] = True$



Fila de prioridade

heapq sempre extrai o vértice de menor distância.



Inicialização

$dist[1]=0$, demaiss ∞ . $via_estrada[1]=True$.

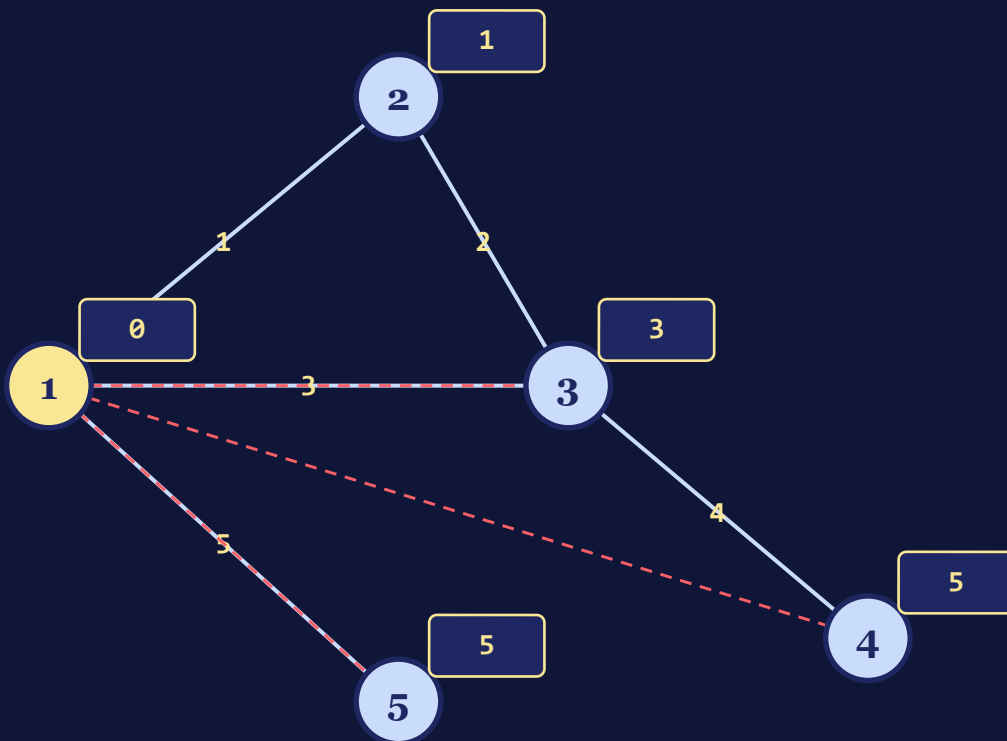


Parada

Fila vazia $\rightarrow dist[]$ final, com trens já incluídos.

5. Dijkstra em ação — Exemplo 1

Distâncias finais (pílulas douradas)



Vetores ao final

índice	1	2	3	4	5
dist	0	1	3	5	5
via_estrada	T	T	T	F	T

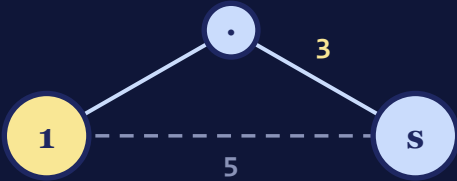
Como ler

- Cidades 2, 3, 5 têm caminho ótimo por estradas.
- Só a cidade 4 depende do trem (via_estrada = F).
- Logo, no exemplo, 2 dos 3 trens podem ser fechados.

6. Quando fechar cada trem?

Trem inútil

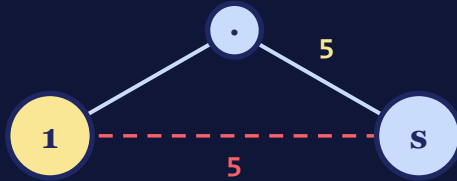
$y > \text{dist}[s]$



× FECHA

Alternativa por estrada

$y = \text{dist}[s]$, via estrada



× FECHA

Trem essencial

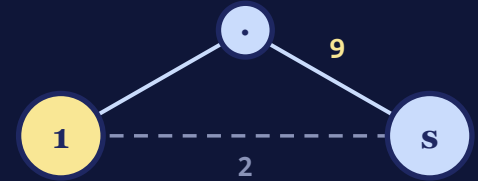
$y = \text{dist}[s]$, sem estrada



✓ MANTÉM 1

Caso impossível

$y < \text{dist}[s]$



— não ocorre

No exemplo 1

trem (3,5): $5 > 3 \rightarrow$ fecha

trem (4,5): $5 = 5$, sem estrada $\rightarrow$ mantém

trem (5,5): $5 = 5$, há estrada $\rightarrow$ fecha

7. Complexidade

Tempo

$$O((V + E) \cdot \log V)$$

$V = n$ *vértices*

$E = 2m + k$ *estradas (×2) + trens*

$\log V$ *heap binário (heapq)*

$O(V+E)$

memória
(adjacência + vetores)

1625 ms

tempo real
(limite 2000 ms)

180 MB

memória real
(limite 256 MB)

8. Casos especiais

10^9

Pesos grandes

soma $\sim 10^{14}$ — int do Python (long em Java).

≡

Trens repetidos

agrupa por destino; mantém só o necessário.

||

Arestas paralelas

cada uma é uma entrada; Dijkstra escolhe.

=

**Empate trem ×
estrada**

via_estrada detecta e fecha o trem.

↓

**Trem melhora
distância**

é só mais uma aresta no Dijkstra.

✓

Sempre conexo

nenhum $\text{dist}[v]$ fica em ∞ .

Resultado

Accepted

#376408159

submissão

1625 ms

tempo (limite 2 s)

180 MB

memória (limite 256 MB)

Em resumo

- Trens são apenas arestas extras a partir da capital.
- Um único Dijkstra + a marca via_estrada já resolve.
- Conta-se direto, por destino, quantos trens podem fechar.